

FoLang complete EBNF — revision 2

This draft consolidates the syntax in `language-ref(32).md` and makes explicit, reviewable decisions where the current reference is incomplete. The decisions are not silent parser assumptions; each is tagged in the grammar and listed in the decision register.

Included files

- `folang-complete-v2.ebnf` — consolidated EBNF.
- `folang-ebnf-decisions.md` — proposed normative decisions and reference-document changes.
- `folang-ebnf-v2-validation.json` — mechanical validation report.

Grammar

(*

FoLang consolidated EBNF – decision-complete draft, revision 2
Derived from language-ref(32).md, 2026-07-28.

Notation:

= defines a production
; ends a production
| alternative
[...] optional
{ ... } zero or more
(...) grouping
"..." terminal text
? ... ? a precisely described lexical or context-sensitive terminal

STATUS

The language reference remains authoritative. Where that document did not select a lexical or parsing rule, this revision makes an explicit design decision based on established C/Java/Rust/Swift-style practice. Every such decision is labelled DECISION-* in this grammar and in the companion decision register so it can be copied into language-ref.md.

PRINCIPAL DECISIONS

DECISION-SYN-001: A semicolon is mandatory after every simple statement and every standalone built-in directive. Newlines never terminate statements and there is no semicolon insertion. A block-bodied declaration is not followed by a semicolon.

DECISION-OP-001: Built-in operators use the precedence table encoded in section 11. Assignment has the lowest built-in precedence.

DECISION-OP-002: Runtime assignment operators are right-associative. Therefore `a = b = c` parses as `a = (b = c)`. An assignment expression yields the assigned value. FoLang's separately specified target-first, left-to-right evaluation order is retained.

DECISION-OP-003: `:=` and `?=` are statement-level definition operators, not general expression operators; they cannot be chained. `::=` remains reserved and is not accepted by this grammar.

DECISION-LEX-001: Source files are UTF-8. Identifiers use Unicode `XID_Start` and `XID_Continue` and are normalized to NFC. Keywords are ASCII, case-sensitive tokens. A lone `_` is the wildcard.

DECISION-LEX-002: `//` and non-nesting `/* ... */` comments are supported. Line breaks are ordinary whitespace outside literals.

DECISION-LEX-003: The lexer applies maximal munch. Reserved multi-character operators are chosen before their shorter prefixes.

DECISION-LIT-001: Integer literals support decimal, `0b` binary, `0o` octal, and `0x` hexadecimal forms with internal underscores. Numeric suffixes are not part of the initial grammar.

DECISION-LIT-002: Floating literals are decimal, require digits after a

decimal point when a point is present, and may use an exponent. Hexadecimal floats are not initially supported.

DECISION-LIT-003: Strings and characters use explicit escapes listed in section 12, including `\\u{...}`. Raw, multiline, and interpolated strings are not initially supported.

DECISION-COL-001: Commas separate enum variants, map entries, annotation-map entries, and object initializers; a trailing comma is allowed. Object and annotation-map fields use colon.

DECISION-BLK-001: A block may end in one unterminated tail expression. That expression is the block value and is not a statement.

DECISION-EXT-001: User-defined operators are parsed by the compiler's registered precedence table. Overloads of built-in symbols retain built-in precedence. New symbols require declared fixity, numeric precedence, and associativity.

Context-sensitive rules such as source-file kind, declaration legality, type checking, visibility, operator ownership, capture, and definite initialization remain semantic constraints and are documented separately.

*)

(* ===== *)

(* 1. Compilation units *)

(* ===== *)

```

compilation-unit = package-source-file
                  | application-entry-file
                  | library-surface-file ;

```

```

package-source-file = file-preamble, primary-declaration ;

```

```

application-entry-file = file-preamble, { entry-item } ;

```

```

library-surface-file = file-preamble, library-declaration ;

```

```

file-preamble = { file-directive } ;

```

```

file-directive = import-directive
                  | alias-directive
                  | use-directive
                  | dynamic-runtime-directive
                  | pragma-directive
                  | generic-directive ;

```

```

entry-item = file-directive
              | entry-type-declaration
              | bare-function-pattern-clause
              | capturing-function-pattern-clause
              | statement ;

```

```

primary-declaration = struct-declaration
                       | cstruct-declaration
                       | enum-declaration

```

```

    | union-declaration
    | data-declaration
    | class-declaration
    | interface-declaration
    | signature-declaration
    | module-declaration
    | unit-declaration
    | type-declaration
    | object-declaration
    | instance-declaration
    | matcher-instance-declaration
    | function-object-declaration
    | delegate-declaration
    | named-block-declaration
    | annotated-contract-declaration
    | annotated-function-primary
    | type-constructor-primary
    | forward-type-declaration
    | package-alias-declaration ;

(* ===== *)
(* 2. Directives, annotations, and metadata *)
(* ===== *)

annotations = { annotation } ;

one-or-more-annotations = annotation, { annotation } ;

annotation = "@", qualified-name,
    [ "(", [ annotation-argument-list ], ")" ] ;

annotation-argument-list = annotation-argument,
    { ",", annotation-argument }, [ "," ] ;

annotation-argument = [ annotation-key, "=" ], annotation-value ;

annotation-key = identifier, { "-", identifier } ;

annotation-value = literal
    | type-expression
    | qualified-name
    | declaration-reference
    | annotation-list
    | annotation-map
    | annotation-arrow-pair ;

annotation-list = "[", [ annotation-value,
    { ",", annotation-value }, [ "," ] ], "]" ;

annotation-map = "{", [ annotation-map-entry,
    { ",", annotation-map-entry }, [ "," ] ], "}" ;

```

```

(* DECISION-COL-001: annotation-map entries use colon, not equals. *)
annotation-map-entry = annotation-key, ":", annotation-value ;

annotation-arrow-pair = string-literal, "=>", string-literal ;

(* DECISION-SYN-001: standalone directives are terminated by semicolons. *)
import-directive = "@co.ddap.import", "(", import-field,
    { ",", import-field }, [ ",", ], ")", statement-end ;

import-field = ( "package" | "library" | "src-library" | "expect"
    | "as" | "realm" | "parent-realm" ), "=", annotation-value ;

alias-directive = "@co.ddap.alias", "(", qualified-name, ",",
    "as", "=", string-literal, [ ",", ], ")",
    statement-end ;

use-directive = "@co.ddap.use", "(", annotation-argument-list, ")",
    statement-end ;

dynamic-runtime-directive = "@co.ddap.dynamicruntime",
    [ "(", [ annotation-argument-list ], ")" ],
    statement-end ;

pragma-directive = ( "@co.pdap.compiler" | "@co.pdap.scale" ),
    [ "(", [ annotation-argument-list ], ")" ],
    statement-end ;

generic-directive = "@co.ddap.", identifier,
    [ "(", [ annotation-argument-list ], ")" ],
    statement-end ;

(* ===== *)
(* 3. Names and references *)
(* ===== *)

declaration-name = identifier | "_" ;

qualified-name = ( identifier | "co" ), { ".", identifier } ;

co-path = "co", ".", identifier, { ".", identifier } ;

declaration-reference = qualified-function-reference | qualified-name ;

qualified-function-reference = qualified-name, "(", [ type-list ], ")",
    return-type-clause ;

lifecycle-name = "@@", identifier ;

special-binding = "$", { digit } ;

result-binding = "$", nonzero-digit, { digit } ;

```

```

wildcard = "_" ;

(* ===== *)
(* 4. Type syntax *)
(* ===== *)

type-expression = forall-type | union-type-expression ;

forall-type = "forall", "(", type-parameter-list, ")", ".", type-expression ;

type-parameter-list = identifier, { ",", identifier } ;

union-type-expression = arrow-type-expression,
                        { "|", arrow-type-expression } ;

arrow-type-expression = type-postfix-expression,
                        [ "->", arrow-type-tail ] ;

arrow-type-tail = type-derivation
                 | parenthesized-type-list
                 | type-expression ;

type-postfix-expression = type-atom, { type-argument-list } ;

type-atom = qualified-name
           | "(", type-expression, ")"
           | "(", type-list, ")" ;

type-argument-list = "(", [ type-or-value-argument,
                           { ",", type-or-value-argument } ], ")" ;

type-or-value-argument = type-expression | constant-expression | identifier ;

type-list = type-expression, { ",", type-expression } ;

parenthesized-type-list = "(", [ type-list ], ")" ;

type-derivation = "(", derivation-specification, ")" ;

derivation-specification = pointer-specification
                          | array-specification
                          | reference-specification
                          | range-type-specification
                          | slice-type-specification
                          | thunk-type-specification
                          | address-type-specification
                          | word-type-specification
                          | derivation-attribute-list ;

pointer-specification = pointer-stars,
                       [ ",", derivation-attribute-list ] ;

```

```

pointer-stars = "*", { "*" } ;

reference-specification = "&" | "&&" | "~" ;

address-type-specification = "@" ;

thunk-type-specification = "^" ;

slice-type-specification = "[:]" ;

range-type-specification = ".." ;

array-specification = array-dimension-group, { array-dimension-group } ;

array-dimension-group = "[", array-dimension-content, "]" ;

array-dimension-content = [ array-dimension,
                           { ",", [ array-dimension ] } ] ;

array-dimension = integer-literal | identifier | "..." | "." | expression ;

word-type-specification = derivation-attribute-list ;

derivation-attribute-list = derivation-attribute,
                           { ",", derivation-attribute } ;

derivation-attribute = annotation-key, "=", annotation-value ;

return-type-clause = "->", "(", [ return-item-list ], ")" ;

return-item-list = return-item, { ",", return-item } ;

return-item = [ identifier ], type-expression ;

(* ===== *)
(* 5. Common declaration components *)
(* ===== *)

generic-parameter-clause = "(", identifier, { ",", identifier }, ")" ;

kind-options = "->", "(", [ annotation-argument-list ], ")" ;

declaration-prefix = annotations ;

field-declaration = annotations, identifier, type-expression,
                   [ "=", expression ], statement-end ;

embedded-field-declaration = annotations, type-expression, statement-end ;

value-specification = annotations, identifier, type-expression, statement-end ;

(* DECISION-SYN-002: comma-separated variable declarations are one statement. *)

```

```

variable-declaration = annotations, typed-variable-declarator,
                        { ",", typed-variable-declarator }, statement-end ;

typed-variable-declarator = identifier, type-expression,
                           [ "=", expression ] ;

inferred-variable-declaration = annotations, inferred-variable-declarator,
                                { ",", inferred-variable-declarator },
                                statement-end ;

inferred-variable-declarator = identifier,
                              ( ":@" | "?@" ), expression ;

external-variable-declaration = annotations, identifier, type-expression,
                                statement-end ;

(* ===== *)
(* 6. Data and type declarations *)
(* ===== *)

struct-declaration = annotations, declaration-name,
                      [ generic-parameter-clause ], "co.lang.struct", "=",
                      struct-body ;

struct-body = "{", { struct-member }, "}" ;

struct-member = field-declaration | embedded-field-declaration ;

cstruct-declaration = annotations, declaration-name,
                      [ generic-parameter-clause ], "co.lang.cstruct", "=",
                      cstruct-body ;

cstruct-body = "{", { field-declaration }, "}" ;

enum-declaration = annotations, declaration-name,
                  [ generic-parameter-clause ], "co.lang.enum", "=",
                  enum-body ;

enum-body = "{", [ enum-variant,
                  { enum-separator, enum-variant }, [ enum-separator ] ], "}" ;

(* DECISION-COL-001: enum variants are comma-separated. *)
enum-separator = "," ;

enum-variant = annotations, identifier,
               [ "(", [ type-list ], ")" ],
               [ "=", constant-expression ] ;

union-declaration = annotations, declaration-name,
                   [ generic-parameter-clause ], "co.lang.union", "=",
                   union-body ;

```



```

union-body = "{", { field-declaration }, "}" ;

data-declaration = annotations, declaration-name,
                  [ generic-parameter-clause ], "co.lang.data", "=",
                  data-variant, { "|", data-variant }, statement-end ;

data-variant = qualified-name,
              [ "(", [ type-list ], ")" ] ;

type-declaration = annotations, declaration-name,
                  [ generic-parameter-clause ], type-declaration-kind,
                  [ "=", type-expression ], statement-end ;

type-declaration-kind = "co.lang.type"
                      | "co.lang.typealias"
                      | "co.lang.newtype"
                      | "co.lang.opaquetype"
                      | "co.lang.subtype"
                      | "co.lang.supertype"
                      | "co.lang.associatedtype"
                      | "co.lang.refinementType" ;

entry-type-declaration = type-declaration ;

forward-type-declaration = annotations, declaration-name,
                          [ generic-parameter-clause ],
                          forward-declarable-kind, [ kind-options ],
                          statement-end ;

forward-declarable-kind = "co.lang.struct"
                        | "co.lang.cstruct"
                        | "co.lang.class"
                        | "co.lang.interface"
                        | "co.lang.signature"
                        | "co.lang.module"
                        | "co.lang.enum"
                        | "co.lang.union"
                        | "co.lang.data"
                        | "co.lang.object"
                        | "co.lang.instance"
                        | "co.lang.function" ;

package-alias-declaration = declaration-name, "co.lang.package", statement-end ;

(* ===== *)
(* 7. Containers and behavioral declarations *)
(* ===== *)

unit-declaration = annotations, declaration-name, "co.lang.unit", "=",
                  unit-body ;

unit-body = "{", { function-declaration }, "}" ;

```

```
class-declaration = annotations, declaration-name,  
                    [ generic-parameter-clause ], "co.lang.class",  
                    [ kind-options ], "=", class-body ;  
  
class-body = "{", { class-member }, "}" ;  
  
class-member = field-declaration  
              | function-declaration  
              | lifecycle-method-declaration ;  
  
lifecycle-method-declaration = annotations, lifecycle-name,  
                                parameter-list, [ return-type-clause ],  
                                function-definition ;  
  
interface-declaration = annotations, declaration-name,  
                        [ generic-parameter-clause ], "co.lang.interface", "=",  
                        interface-body ;  
  
interface-body = "{", { function-specification }, "}" ;  
  
signature-declaration = annotations, declaration-name,  
                        [ generic-parameter-clause ], "co.lang.signature", "=",  
                        signature-body ;  
  
signature-body = "{", { signature-member }, "}" ;  
  
signature-member = value-specification  
                  | function-specification  
                  | signature-type-component ;  
  
signature-type-component = annotations, declaration-name,  
                            [ generic-parameter-clause ], "co.lang.type",  
                            [ "=", type-expression ], statement-end ;  
  
module-declaration = annotations, declaration-name, "co.lang.module",  
                    [ kind-options ], "=", module-body ;  
  
module-body = "{", { module-member }, "}" ;  
  
module-member = variable-declaration  
               | inferred-variable-declaration  
               | function-declaration  
               | signature-type-component ;  
  
library-declaration = annotations, declaration-name, "co.lang.library", "=",  
                    library-body ;  
  
library-body = "{", { library-member }, "}" ;  
  
library-member = import-directive  
               | struct-declaration
```

```

    | cstruct-declaration
    | function-declaration ;

object-declaration = annotations, declaration-name, "co.lang.object",
    [ kind-options ], "=", object-body ;

object-body = "{", { field-declaration | function-declaration }, "}" ;

instance-declaration = annotations, declaration-name, "co.lang.instance",
    [ kind-options ], "=", instance-body ;

instance-body = "{", { function-declaration | variable-declaration }, "}" ;

matcher-instance-declaration = annotations, declaration-name,
    ( "co.lang.Matcher" | "co.lang.matcher" ),
    [ kind-options ], "=", instance-body ;

annotated-contract-declaration = one-or-more-annotations, declaration-name,
    [ generic-parameter-clause ], "=",
    contract-body ;

contract-body = "{", { function-specification | value-specification }, "}" ;

named-block-declaration = annotations, declaration-name, "co.lang.block", "=",
    block ;

delegate-declaration = annotations, declaration-name, "co.lang.delegate", "=",
    function-type, statement-end ;

function-object-declaration = annotations, declaration-name,
    "co.lang.function", "=",
    anonymous-function-expression,
    statement-end ;

annotated-function-primary = one-or-more-annotations, function-declaration ;

type-constructor-primary = annotations, function-name, parameter-list,
    { parameter-list }, return-type-clause,
    function-binding ;

(* ===== *)
(* 8. Functions *)
(* ===== *)

function-declaration = annotations, [ receiver-clause ], function-name,
    parameter-list, { parameter-list },
    [ return-type-clause ], function-binding ;

function-name = identifier | lifecycle-name ;

receiver-clause = "(", ( type-expression
    | identifier, type-expression ), ")" ;

```

```

parameter-list = "(", [ parameter,
                    { ",", parameter }, [ ",", ] ], ")" ;

parameter = [ "...", [ "~" ], identifier, [ "?" ],
             [ type-expression ], [ "=", expression ] ;

function-binding = function-definition
                  | function-delegation
                  | function-alias-binding
                  | statement-end ;

function-definition = "=", block ;

function-delegation = ( ">" | ">>" ), expression,
                      { ">>", expression }, statement-end ;

function-alias-binding = "=", expression, statement-end ;

function-specification = annotations, [ receiver-clause ], function-name,
                           parameter-list, { parameter-list },
                           [ return-type-clause ], statement-end ;

function-type = "(", [ type-list ], ")", return-type-clause ;

anonymous-function-expression = [ "forall", "(", type-parameter-list, ")", "." ],
                                parameter-list, return-type-clause,
                                [ "=" ], block ;

lambda-expression = "|", [ lambda-parameter,
                           { ",", lambda-parameter } ], "|", ">",
                           ( expression | block ) ;

lambda-parameter = identifier, [ type-expression ] ;

(* ===== *)
(* 9. Function-pattern groups and patterns *)
(* ===== *)

bare-function-pattern-clause = identifier, pattern-parameter-list,
                              [ where-clause ], ">", pattern-result ;

capturing-function-pattern-clause = "let", identifier,
                                   pattern-parameter-list,
                                   [ where-clause ], "=", pattern-result ;

pattern-parameter-list = "(", [ pattern,
                              { ",", pattern } ], ")" ;

where-clause = ".where", "(", expression, ")" ;

pattern-result = expression | block ;

```

```

pattern = wildcard
    | literal-pattern
    | binding-pattern
    | constructor-pattern
    | record-pattern
    | tuple-pattern
    | qualified-name ;

literal-pattern = literal ;

binding-pattern = identifier ;

constructor-pattern = qualified-name, "(", [ pattern,
    { ",", pattern } ], ")" ;

record-pattern = qualified-name, "{", [ record-pattern-field,
    { ",", record-pattern-field } ], "}" ;

record-pattern-field = identifier, [ ":", pattern ] ;

tuple-pattern = "(", pattern, ",", pattern,
    { ",", pattern }, ")" ;

match-case = ".case", "(", match-case-body, ")" ;

match-case-body = pattern, [ ":", expression ], "=>",
    ( expression | block ) ;

match-default = ".default", "(", ( expression | block ), ")" ;

(* ===== *)
(* 10. Statements and blocks *)
(* ===== *)

(*
    DECISION-SYN-001:
    - Every simple statement ends with ";".
    - A newline is whitespace and never terminates a statement.
    - A block statement and a block-bodied declaration do not take a trailing
      semicolon merely because their final token is "}".

    DECISION-BLK-001:
    A final expression without a semicolon is a block tail expression, not an
    expression statement. It supplies the block's value.
*)
block = "{", { block-item }, [ block-tail-expression ], "}" ;

block-item = statement ;

block-tail-expression = expression ;

```

```

statement = variable-declaration
           | inferred-variable-declaration
           | grouped-variable-declaration
           | let-value-declaration
           | multiple-assignment-statement
           | return-statement
           | expression-statement
           | labeled-block
           | empty-statement ;

grouped-variable-declaration = "(", typed-variable-declarator,
                               { ",", typed-variable-declarator }, ")",
                               statement-end ;

let-value-declaration = "let", identifier, [ type-expression ], "=",
                        expression, statement-end ;

(*
  DECISION-OP-003:
  := and ?= occur only in inferred-variable-declaration. They are not
  assignment-expression operators and cannot participate in a = b = c-style
  chain. ::= remains reserved for a future feature and is rejected.
*)

(* Multiple assignment is a statement because it has multiple destinations. *)
multiple-assignment-statement = assignment-target, ",", assignment-target,
                                { ",", assignment-target }, "=",
                                expression-list, statement-end ;

assignment-target = postfix-expression
                  | tuple-assignment-target ;

tuple-assignment-target = "(", assignment-target, ",", assignment-target,
                          { ",", assignment-target }, ")" ;

return-statement = ( "this" | "self" ), ".return",
                    [ expression-list ], statement-end ;

expression-statement = expression, statement-end ;

labeled-block = identifier, ":", block ;

empty-statement = ";" ;

expression-list = expression, { ",", expression } ;

statement-end = ";" ;

(* ===== *)
(* 11. Expressions and built-in operator precedence *)
(* ===== *)

```

```

(*)
  DECISION-OP-001 – built-in precedence, highest to lowest:

  100 postfix: calls, indexing, member access, postfix !, ++, -- left
  90  exponentiation: ** right
  80  prefix: +, -, !, ~, @, #, ^, ++, -- right
  70  multiplicative: *, /, % left
  60  additive: +, - left
  55  ranges: .., <.., ..<, <..< none
  50  relational: <, <=, >, >= left
  45  equality: ==, != left
  40  bitwise AND: & left
  38  bitwise XOR: ^ left
  36  bitwise OR: | left
  30  logical AND: && left
  20  logical OR: || left
  10  assignment: =, +=, -=, *=, /=, %=, **=, &=, ^=, |= right

  Operands are still evaluated according to FoLang's normative left-to-right
  and target-first evaluation rules. Associativity determines grouping, not
  the order in which operand subexpressions are evaluated.

  DECISION-EXT-001:
  A use of at least one newly defined custom operator is parsed by the
  registered operator table. The contextual production extended-operator-
  expression denotes that compiler-generated precedence grammar. Overloads of
  built-in symbols use the built-in table above and do not alter precedence.
*)

expression = assignment-expression
            | extended-operator-expression ;

(* DECISION-OP-002: right recursion makes assignment right-associative. *)
assignment-expression = logical-or-expression,
                      [ runtime-assignment-operator,
                        assignment-expression ] ;

runtime-assignment-operator = "="
                           | compound-assignment-operator ;

compound-assignment-operator = "+=" | "-=" | "*=" | "/=" | "%="
                             | "**=" | "&=" | "^=" | "|=" ;

constant-expression = logical-or-expression ;

logical-or-expression = logical-and-expression,
                      { "||", logical-and-expression } ;

logical-and-expression = bitwise-or-expression,
                      { "&&", bitwise-or-expression } ;

bitwise-or-expression = bitwise-xor-expression,
                      { "|", bitwise-xor-expression } ;

```

```

bitwise-xor-expression = bitwise-and-expression,
                        { "^", bitwise-and-expression } ;

bitwise-and-expression = equality-expression,
                        { "&", equality-expression } ;

equality-expression = relational-expression,
                    { equality-operator, relational-expression } ;

equality-operator = "==" | "!=" ;

relational-expression = range-expression,
                    { relational-operator, range-expression } ;

relational-operator = "<" | "<=" | ">" | ">=" ;

(* A range expression contains at most one range operator. *)
range-expression = additive-expression,
                [ range-operator, [ additive-expression ] ]
                | range-operator, additive-expression ;

range-operator = ".." | "<.." | "..<" | "<..<" ;

additive-expression = multiplicative-expression,
                    { additive-operator, multiplicative-expression } ;

additive-operator = "+" | "-" ;

multiplicative-expression = unary-expression,
                        { multiplicative-operator, unary-expression } ;

multiplicative-operator = "*" | "/" | "%" ;

unary-expression = { prefix-operator }, power-expression ;

(* DECISION-OP-004: ++ and -- support both prefix and postfix forms. *)
prefix-operator = "+" | "-" | "!" | "~" | "@" | "#" | "^"
                | "++" | "--" ;

(* Right recursion makes exponentiation right-associative. *)
power-expression = postfix-expression,
                [ "**", unary-expression ] ;

postfix-expression = primary-expression,
                    { postfix-suffix | postfix-operator } ;

postfix-operator = "!" | "++" | "--" ;

postfix-suffix = call-suffix
                | index-suffix
                | member-suffix

```



```

    | match-suffix ;

call-suffix = "(", [ argument-list ], ")" ;

argument-list = argument, { ",", argument }, [ ",", ] ;

argument = ( [ identifier, "=" ], expression )
    | block
    | lambda-expression ;

index-suffix = "[", [ expression-list ], "]" ;

member-suffix = ".", ( identifier | lifecycle-name ) ;

member-access-expression = primary-expression, member-suffix,
    { member-suffix | call-suffix | index-suffix } ;

index-expression = primary-expression, index-suffix,
    { member-suffix | call-suffix | index-suffix } ;

primary-expression = literal
    | special-binding
    | "this"
    | "self"
    | qualified-name
    | wildcard
    | grouped-expression
    | tuple-expression
    | array-literal
    | map-literal
    | object-construction
    | anonymous-class-expression
    | block
    | anonymous-function-expression
    | lambda-expression
    | let-expression
    | comprehension-expression ;

grouped-expression = "(", expression, ")" ;

tuple-expression = "(", expression, ",", expression,
    { ",", expression }, [ ",", ], ")" ;

array-literal = "[", [ expression,
    { ",", expression }, [ ",", ] ], "]" ;

map-literal = "{", [ map-entry,
    { ",", map-entry }, [ ",", ] ], "}" ;

map-entry = expression, ":", expression ;

(* DECISION-COL-001: object fields use colon and comma. *)

```

```

object-construction = type-postfix-expression, "{",
                        [ object-field-initializer,
                          { ",", object-field-initializer }, [ ",", " ] ], "}" ;

object-field-initializer = identifier, ":", expression ;

anonymous-class-expression = "co.lang.class", "{",
                              { class-member }, "}" ;

let-expression = "let", "(", "{", let-binding,
                 { ",", let-binding }, ")", ")",
                 ".in", "(", "{", expression, ")", ")" ;

let-binding = ( identifier | special-binding ), "=", expression ;

comprehension-expression = "for", "(", comprehension-binding, ")",
                           ".yield", "(", expression-list, ")" ;

comprehension-binding = pattern, "<-", expression ;

match-suffix = ".match", [ "(", [ expression ], ")" ],
               match-case, { match-case }, [ match-default ] ;

extended-operator-expression =
    ? expression containing a registered non-built-in operator, parsed by
      precedence climbing from its declared fixity, precedence, associativity,
      and arity; all built-in subexpressions obey the table above ? ;

(* ===== *)
(* 12. Literals and lexical grammar *)
(* ===== *)

(*
  DECISION-LEX-001:
  Source text is UTF-8. A U+FEFF byte-order mark is permitted only as the
  first code point and is otherwise an error. Identifiers are normalized to
  Unicode NFC before equality and reserved-word checks.

  DECISION-LEX-002:
  Horizontal whitespace, line terminators, line comments, and non-nesting
  block comments are discarded between tokens. A line terminator has no
  statement-termination meaning.

  DECISION-LEX-003:
  Lexing uses maximal munch. For example, <.< is selected before <.. and <,
  **= before ** and *, and =>> before => and =. A comment introducer is
  recognized before the / operator.
*)

literal = integer-literal
         | floating-literal
         | string-literal

```

```

    | character-literal
    | boolean-literal ;

(* DECISION-LIT-001: no numeric type suffixes in the initial grammar. *)
integer-literal = binary-integer-literal
    | octal-integer-literal
    | hexadecimal-integer-literal
    | decimal-integer-literal ;

binary-integer-literal = "0", ( "b" | "B" ), binary-digit-sequence ;

octal-integer-literal = "0", ( "o" | "O" ), octal-digit-sequence ;

hexadecimal-integer-literal = "0", ( "x" | "X" ),
    hexadecimal-digit-sequence ;

decimal-integer-literal = "0" | nonzero-digit, { [ "_" ], decimal-digit } ;

binary-digit-sequence = binary-digit, { [ "_" ], binary-digit } ;

octal-digit-sequence = octal-digit, { [ "_" ], octal-digit } ;

hexadecimal-digit-sequence = hexadecimal-digit,
    { [ "_" ], hexadecimal-digit } ;

(*
    DECISION-LIT-002:
    A decimal point must be followed by at least one digit. Consequently 1..10
    tokenizes as integer 1, range operator .., integer 10, never as 1. plus .10.
*)
floating-literal = decimal-digit-sequence, ".", decimal-digit-sequence,
    [ exponent-part ]
    | decimal-digit-sequence, exponent-part ;

decimal-digit-sequence = decimal-digit, { [ "_" ], decimal-digit } ;

exponent-part = ( "e" | "E" ), [ "+" | "-" ],
    decimal-digit-sequence ;

(* DECISION-LIT-003: regular single-line strings only in the initial grammar. *)
string-literal = double-quote,
    { string-character | escape-sequence }, double-quote ;

character-literal = single-quote,
    ( character-character | escape-sequence ), single-quote ;

string-character = ? any Unicode scalar value except double quote, backslash,
    carriage return, or line feed ? ;

character-character = ? any Unicode scalar value except single quote,
    backslash, carriage return, or line feed ? ;

```

```

escape-sequence = backslash,
    ( "0" | "b" | "t" | "n" | "f" | "r"
    | double-quote | single-quote | backslash
    | unicode-escape ) ;

unicode-escape = "u", "{", hexadecimal-digit,
    [ hexadecimal-digit ], [ hexadecimal-digit ],
    [ hexadecimal-digit ], [ hexadecimal-digit ],
    [ hexadecimal-digit ], "}" ;

double-quote = ? Unicode scalar value U+0022 ? ;

single-quote = ? Unicode scalar value U+0027 ? ;

backslash = ? Unicode scalar value U+005C ? ;

boolean-literal = "true" | "false" ;

(
    DECISION-LEX-001:
    identifier-start and identifier-continue follow Unicode Standard Annex #31.
    The spelling "_" alone is tokenized as wildcard, not as identifier.
*)
identifier = identifier-start, { identifier-continue } ;

identifier-start = "_" | unicode-xid-start ;

identifier-continue = "_" | unicode-xid-continue ;

unicode-xid-start = ? Unicode code point with the XID_Start property ? ;

unicode-xid-continue = ? Unicode code point with the XID_Continue property ? ;

binary-digit = "0" | "1" ;

octal-digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" ;

hexadecimal-digit = decimal-digit
    | "a" | "b" | "c" | "d" | "e" | "f"
    | "A" | "B" | "C" | "D" | "E" | "F" ;

digit = decimal-digit ;

decimal-digit = "0" | nonzero-digit ;

nonzero-digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

hard-reserved-word = "co" | "let" | "this" | "for" | "forall" | "fo"
    | "true" | "false" ;

contextual-keyword = "self" ;

```

```

line-comment = "//", { ? any Unicode scalar value except CR or LF ? } ;

block-comment = "/*", { block-comment-character }, "*/" ;

block-comment-character = ? any Unicode scalar value that does not begin the
                           two-character sequence */ ? ;

line-break = "\r\n" | "\n" | "\r" ;

horizontal-white-space = " " | "\t" | "\f" ;

white-space = horizontal-white-space | line-break | line-comment | block-comment ;

```

FoLang EBNF decision register — revision 2

This register lists the rules added where `language-ref(32).md` did not yet make a complete lexical or parsing decision. These are proposed normative rules, not hidden assumptions. The matching `DECISION-*` identifiers also appear as comments in `folang-complete-v2.ebnf`.

1. Statement termination

DECISION-SYN-001 — mandatory semicolons

- Every **simple statement** ends with `;`.
- Every standalone built-in directive such as `@co.ddap.import(...)`, `@co.ddap.alias(...)`, and `@co.ddap.use(...)` ends with `;`.
- Newlines are whitespace only. FoLang performs no automatic semicolon insertion.
- A block statement and a block-bodied declaration do not require a second semicolon after their closing `}`.
- The complete condition/loop/iterator chain is an expression statement and therefore ends with `;`.

Examples:

```

value := calculate();
value = other = source;

condition.do({
    run();
}).otherwise.do({
    recover();
});

Employee co.lang.struct = {
    id co.lang.int;
    name co.lang.string;
}

```

The struct declaration itself has no semicolon after `}`; its field declarations do.

DECISION-BLK-001 — block tail expressions

A final expression without `;` is a block's value-producing tail expression. It is not a statement and therefore does not violate mandatory statement termination.

```
classify(n) => {
  n + 1
}
```

Writing `n + 1;` makes it an expression statement and the block has no tail value.

2. Assignment and operators

DECISION-OP-001 — built-in precedence

Precedence	Operators/forms	Associativity
100	call <code>()</code> , index <code>[]</code> , member <code>.</code> , postfix <code>!</code> , <code>++</code> , <code>--</code>	left
90	<code>**</code>	right
80	prefix <code>+</code> , <code>-</code> , <code>!</code> , <code>~</code> , <code>@</code> , <code>#</code> , <code>^</code> , <code>++</code> , <code>--</code>	right
70	<code>*</code> , <code>/</code> , <code>%</code>	left
60	<code>+</code> , <code>-</code>	left
55	<code>..</code> , <code><..</code> , <code>..<</code> , <code><..<</code>	non-associative
50	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	left
45	<code>==</code> , <code>!=</code>	left
40	<code>&</code>	left
38	<code>^</code>	left
36	<code>`</code>	<code>`</code>
30	<code>&&</code>	left, short-circuit
20	<code>`</code>	
10	assignment operators	right

Precedence determines grouping. It does not replace FoLang's normative left-to-right operand evaluation or target-first assignment evaluation.

Exponentiation binds more tightly than prefix operators, so `-2 ** 2` means `-(2 ** 2)`. Both prefix and postfix forms of `++` and `--` are accepted. Prefix form mutates and yields the new value; postfix form yields the previous value and then mutates.

DECISION-OP-002 — right-associative assignment

```
a = b = c;
```

is parsed as:

```
a = (b = c);
```

An assignment expression yields the value assigned. The assignment target is still resolved/evaluated before its right-hand side, as required by the existing evaluation-order section.

Runtime assignment operators are:

```
= += -= *= /= %= **= &= ^= |=
```

DECISION-OP-004 — increment and decrement fixity

`++` and `--` are accepted in both prefix and postfix positions, following mainstream C/Java behavior. Prefix form mutates the target and yields the new value. Postfix form yields the previous value and then mutates the target. Their operand must be a mutable assignable target.

DECISION-OP-003 — definition operators are statement-only

- `:=` declares and initializes an inferred variable and errors when the name already exists.
- `?=` declares and initializes when absent; otherwise it reassigns the existing binding.
- Neither operator is a general expression operator, returns a chainable assignment value, or may appear in `a = b := c`.
- `::=` remains reserved until its semantics are defined and is rejected by this grammar.

DECISION-EXT-001 — custom operators

- Overloading a built-in symbol retains that symbol's built-in fixity, precedence, and associativity.
- Defining a new symbol requires explicit `fixity`, numeric `precedence`, `associativity`, and `arity` metadata.
- Higher numeric precedence binds more tightly.
- Associativity is `left`, `right`, or `none`.
- The compiler collects operator declarations before parsing operator expressions, so use is not source-order dependent.
- Maximal munch selects the longest valid operator token.
- A custom operator may not redefine structural delimiters, comment delimiters, arrows, assignment/definition operators, or other reserved grammar tokens.

Because arbitrary registered operators make the expression grammar depend on declarations, their exact expansion is context-sensitive. The EBNF includes a named contextual production for this compiler-generated precedence grammar rather than pretending that a fixed context-free production can enumerate all future symbols.

3. Lexical contract

DECISION-LEX-001 — source encoding and identifiers

- Source files are UTF-8.
- An optional U+FEFF BOM is allowed only at the start of the file.
- Invalid UTF-8 is a lexical error.
- Identifiers follow Unicode `XID_Start` and `XID_Continue`, with `_` also allowed.
- Identifiers are normalized to NFC before comparison and symbol-table insertion.
- A lone `_` is the wildcard/discard token, not an identifier.

- Keywords are case-sensitive ASCII spellings.
- `self` remains contextual; the other listed reserved words are hard-reserved.

DECISION-LEX-002 — whitespace and comments

- Spaces, tabs, form feeds, and line terminators separate tokens.
- `/**` introduces a line comment.
- `/* ... */` introduces a non-nesting block comment.
- Documentation comments are lexically comments until a separate documentation model is specified.
- Newlines never terminate statements.

DECISION-LEX-003 — token selection

The lexer uses maximal munch. Reserved multi-character tokens are chosen before shorter prefixes. Examples:

```
<..<  before  <.. or <
**=    before  ** or *
=>>    before  => or =
..

```

Comment openers are recognized before `/` is treated as an arithmetic operator.

4. Literals

DECISION-LIT-001 — integer literals

Supported:

```
0
42
1_000_000
0b1010_0110
0o755
0xCAFE_BABE
```

Underscores may occur only between digits. Numeric suffixes are excluded initially; explicit types, casts, or contextual typing select the required numeric type.

DECISION-LIT-002 — floating literals

Supported decimal forms include:

```
1.0
3.141_592
1.0e10
6.02e-23
10e3
```

A decimal point must be followed by digits. Therefore `1..10` is unambiguously an integer, a range operator, and another integer. Hexadecimal floating literals are excluded initially.

DECISION-LIT-003 — strings and characters

The initial grammar supports single-line quoted strings and one-scalar character literals. Escapes are:

```
\0 \b \t \n \f \r \" \' \\ \u{HEX}
```

The Unicode escape accepts one to six hexadecimal digits and must denote a Unicode scalar value. Raw strings, multiline strings, and interpolation are excluded until separately specified.

5. Collection and metadata punctuation

DECISION-COL-001 — canonical separators

- Enum variants use commas, with an optional trailing comma.
- Array and tuple elements use commas.
- Map entries use `key: value` and commas.
- Object constructors use `field: value` and commas.
- Annotation maps use `key: value` and commas.
- Named annotation arguments continue to use `name=value`.
- `=` inside an object initializer or annotation map is rejected rather than treated as an alternative spelling.

Example:

```
employee := Employee{
  id: 1001,
  name: "Rao",
};
```

6. Ambiguity-resolution rules

1. `{ ... }` is parsed according to its syntactic position: declaration/block body, map literal, annotation map, or object-construction body.
2. `|x| => expression` starts a lambda only where a primary expression may begin; infix `|` is otherwise bitwise OR.
3. A qualified name followed by `{` is object construction only when the name resolves in type position; otherwise the construct is rejected rather than reinterpreted as a block.
4. Type argument/derivation syntax is parsed in type positions; call syntax is parsed in expression positions.
5. Range operators are non-associative. `a..b..c` is a syntax error unless parentheses create nested range expressions explicitly.
6. Collection/map/object trailing commas are accepted; trailing semicolons are not substitutes for commas.
7. Planned constructs, including comprehensions and package aliasing, remain feature-gated even when represented in the grammar.

7. Existing contextual rules retained

The generated grammar does not weaken the reference's contextual rules:

- ordinary package files contain exactly one primary top-level declaration;
- entry files permit their restricted entry-local declarations and executable statements;
- library surfaces permit only boundary declarations and adapter functions;
- units contain functions and companion ownership is checked semantically;

- no physical local/nested named declarations are introduced by the grammar;
- `forall(T).type` remains type-position syntax, while named generics use annotations;
- capability, visibility, realm, capture, and definite-initialization rules remain semantic checks.

8. Reference-document edits implied by these decisions

Before declaring the grammar normative, update `language-ref.md` to:

1. add the lexical contract and literal sections above;
2. add the built-in precedence table;
3. state assignment's right associativity and value result;
4. state the mandatory-semicolon/no-ASI rule;
5. normalize examples by adding missing semicolons to simple statements and standalone directives;
6. replace enum semicolon separators with commas;
7. replace `=` with `:` inside object initializers and annotation maps;
8. mark `::=`, raw/multiline/interpolated strings, hex floats, and distinct set-literal syntax as unsupported or reserved;
9. document custom-operator collection and precedence integration.